

Teaching a memory without retraining a network



Teach the lab that a handwritten 1 means ALPHA, a 4 means GAMMA and an 8 means BETA. It can then assign those symbols to different images of the same digits. The interesting observation is not simply that a prediction changed. It is that the temporary memory changed while every encoder tensor stayed identical.

MemoryForge turns that distinction into a small experiment. Its controls write real associations, query held-out images, introduce contradictory demonstrations and remove the episode's memory. The charts expose the matrix used for retrieval, alongside the prediction and ground truth. This article explains what those actions establish, why the memory rule works, and where its evidence stops. The implementation and recorded observations are available together. [5]

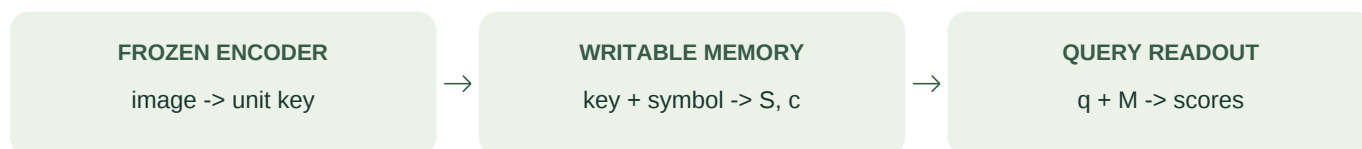
First, separate recognition from association

The lab does not learn handwriting from scratch when a user clicks Teach. A small neural network has already been trained on 1,347 of the bundled scikit-learn digits images. Pixels are divided by 16, flattened into 64 features and passed through layers of sizes 64, 32 and 16, with a ReLU between the two linear layers. A separate ten-class head supplies the supervised training objective.

Training uses 100 epochs of Adam on CPU with fixed seeds. The stored checkpoint's ordinary digit-classification sanity score is 437 correct out of 450 held-out images, or 97.11%. This checks the learned representation; it is not the score for learning arbitrary episode labels. During the interactive experiment, the encoder is frozen and the digit head does not select ALPHA, BETA or GAMMA. [5]

An episode chooses three digit identities and permutes their symbolic labels. Ten candidate demonstrations per class and ten queries per class come from the held-out partition. Their original row IDs are checked for overlap. Increasing the demonstration count selects a longer prefix of the same support pool, keeping the mapping and query images fixed. That pairing makes the effect of additional writes interpretable.

Two stores, two responsibilities



Keys and queries share the encoder. Only demonstrations write to memory.

The encoder supplies a reusable representation. Temporary memory binds that representation to the current symbol. An unseen query image uses the same frozen encoder and reads the memory; the query never becomes a demonstration. New Episode replaces the symbol mapping and starts fresh memory. Clearing memory removes its writes and counts, while preserving the trained encoder.

This is a distinction about state, not a claim that ordinary Transformers cannot adapt to context. Their attention computes query-key affinities and combines values without requiring another training run. [1] MemoryForge makes a particular associative update small enough to inspect numerically. Its educational value comes from exposing where information goes and how it affects retrieval.

A write changes sums; retrieval uses averages

Each support image becomes a unit embedding k with 16 components. Its temporary symbol becomes a one-hot vector v with three components. The memory has a 3-by-16 sum matrix S and a length-three count vector c . The one-hot value selects which sum row receives the key:

```
S <- S + v k^T      c <- c + v
M[i] = S[i] / c[i]  for written labels
scores = M q        q is a unit query key
```

The matrix M therefore contains a mean key for each written symbol. For label i , the score equals the average dot product between the query and keys assigned to i . Both incoming keys and queries are normalized in float64 inside memory, after the encoder's float32 normalization. Class means are not normalized again. This detail matters: replacing the rule with a normalized prototype would produce different scores. [5]

A two-dimensional illustration makes the averaging visible. Suppose the keys (1, 0) and (0, 1) both receive ALPHA. Their sum is (1, 1), their count is two and their mean is (0.5, 0.5). Against unit query (0.8, 0.6), ALPHA scores 0.7. Using the unaveraged sum would give 1.4. This is a hand-calculated illustration of the rule, not a measured digit episode.

The largest score among written labels determines the prediction. Empty memory returns an abstention rather than an arbitrary winning symbol. Exact ties are flagged and resolved deterministically. Raw association scores are not calibrated probabilities; a score of 0.7 does not mean a 70% chance of correctness.

Follow an actual failure

Seed 1000 starts with one support per class. Teaching a second round improves its fixed-query accuracy, but a contradictory write can undo part of that gain. The corruption takes an existing support key and appends it under the next cyclic label. The original ground truth remains unchanged. [5]

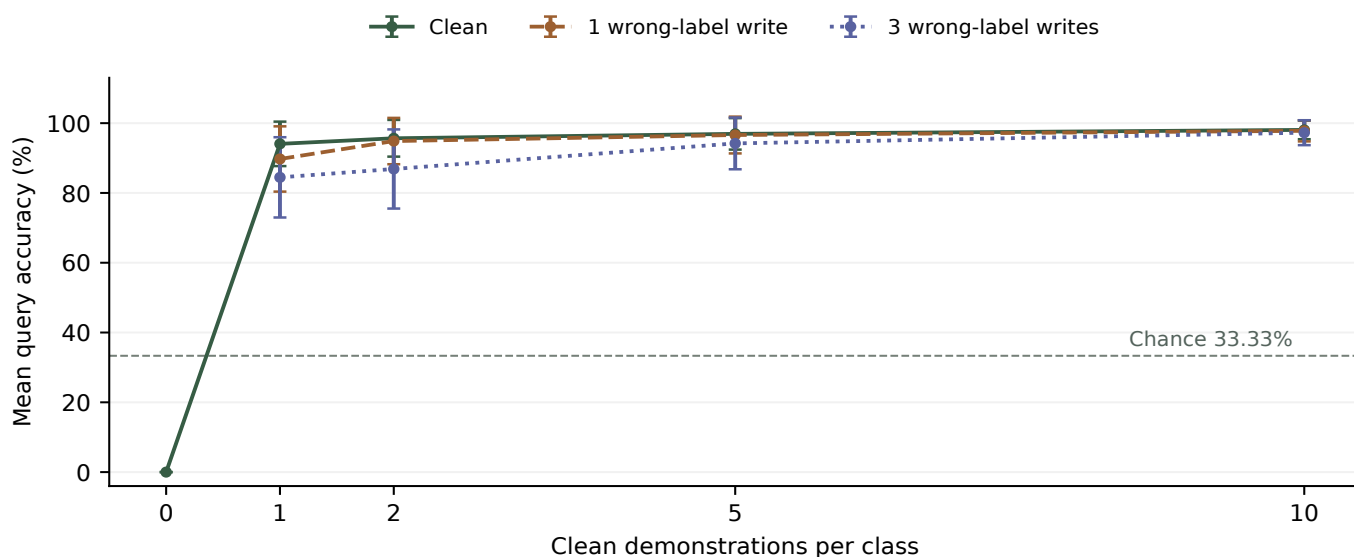
Seed 1000 state	Writes	Correct / 30
One clean demonstration per class	3	23 (76.7%)
Two clean demonstrations per class	6	27 (90.0%)
Append one wrong-label write	7	26 (86.7%)
Clear memory	0	0 (abstaining)

Five predictions change after the conflict, although net accuracy falls by only one correct answer. A changed state need not lower accuracy on every seed, and a changed prediction need not be newly wrong. The lab reports both the transition and the resulting correctness rather than forcing a predetermined failure story.

The displayed memory delta is the Frobenius norm of M relative to empty memory. It can shrink as more keys are averaged, even though more information has been written. Inspecting sums, counts and the effective matrix together prevents that norm from being mistaken for a measure of knowledge. Exact tensor comparison, rather than a frozen-gradient flag alone, checks the separate claim that encoder parameters stayed unchanged.

Challenge the preset with 50 seeds

A single attractive episode cannot establish typical behavior. The saved evidence suite evaluates seeds 1000 through 1049 with 0, 1, 2, 5 and 10 clean demonstrations per class. At each nonzero count it records clean memory, one appended wrong-label write and three appended wrong-label writes. Each condition reads the same 30 query images within its seed. Zero shots supplies the clean abstention condition only. [5]



Recorded CPU evaluation. Error bars show population standard deviation across 50 episode accuracies. [5]

Clean shots/class	Clean mean +/- SD	Three wrong writes: mean +/- SD
1	94.07% +/- 6.37 pp	84.47% +/- 11.50 pp
2	95.67% +/- 5.26 pp	86.87% +/- 11.34 pp
5	96.93% +/- 4.51 pp	94.20% +/- 7.42 pp
10	98.07% +/- 2.67 pp	97.27% +/- 3.57 pp

The zero-shot score is zero because memory abstains. The 33.33% chance reference describes uniform guessing among three symbols. Clean means improve across the displayed counts, but individual episodes need not improve monotonically. Seed 1000, for example, scores 90.0% at two clean shots and 73.3% at five; that decrease is preserved in the raw report.

These are 650 condition evaluations and 19,500 query outcomes, not 19,500 independent trials. Conditions share images, and different seeds can reuse held-out rows. Error bars show population standard deviations of 50 episode accuracies, not confidence intervals. Three extra wrong writes also represent different contamination fractions at different clean shot counts. Their shrinking average impact should not be read as a controlled comparison at a fixed label-noise rate.

Every recorded encoder delta is zero, and two complete reports match exactly. The artifact retains row IDs, mappings, keys, sums, counts, matrices, raw scores, predictions, ties and reset checks. Independent NumPy calculations reconstruct the delivered observations. This is reproducible developer-produced evidence, not an external replication or a deployment study. A fresh CPU installation passed the 124-test suite; those checks establish implementation behavior rather than learner effectiveness. [5]

Connect the mechanism to research

BDH separates fixed graph connections from Hebbian synaptic state. That distinction supports MemoryForge's teaching question, but the toy lacks BDH's graph dynamics and sequence architecture. Shared use of changing state does not establish architectural equivalence. [3]

BDH-CQ separates demonstration-conditioned memory from a recurrent workspace used to compute an answer. MemoryForge implements neither that workspace nor the reported reasoning system; it reads its label matrix once per query. The CQ report leaves exact updates and dimensions proprietary. [4]

DeltaNet uses retrieval-error correction rather than purely additive writing. Its authors' MAD results illustrate selective strengths: In-Context Recall reaches 100%, while Memorize reaches 52.8%, against Transformer baselines of 94.1% and 85.2%. The comparison baselines are borrowed from earlier work. This is a synthetic benchmark, not a deployment or independent reproduction of DeltaNet. [2]

Titans provides a further distinction: its neural memory uses test-time optimization, with momentum and forgetting. Its memory-module parameters can change at test time. MemoryForge performs no adaptation-time gradient update, so the phrase "frozen encoder" must not be generalized into a claim that every test-time memory system freezes all parameters. [6]

Reproduce it and ask the next question

After installing the repository's dependencies, run the app and evidence scripts:

```
python -m streamlit run app.py
python scripts/evaluate_suite.py
python scripts/verify_robustness.py
```

The app loads the supplied checkpoint; it never trains silently. Guided lab exposes the immediate preset, Playground keeps queries fixed while shots change, and Research & evidence links the derivation to the source ledger and saved experiments. Images and frozen embeddings are cached; writes, queries and displayed current memory are computed from the active session.

The most important boundary is the representation. It was trained on familiar digit identities, and the tiny held-out images do not establish performance on unfamiliar categories, language or long-term memory. The system also lacks capacity studies, learned forgetting, consolidation and a human learning evaluation. A useful next experiment would hold the corruption fraction fixed while varying memory size or update rule. The present contribution is narrower and testable: a learner can inspect a real temporary association, challenge it, clear it and verify that the neural encoder did not change.

Sources and technical ownership

[1] Vaswani et al. *Attention Is All You Need* (2017), section 3.2. arXiv:1706.03762v7.

[2] Yang et al. *Parallelizing Linear Transformers with the Delta Rule over Sequence Length* (NeurIPS 2024), sections 2.2, 4.1; Table 1. arXiv:2406.06484v3.

[3] Kosowski et al. *The Dragon Hatchling* (2025 preprint), sections 1.2, 2.2. arXiv:2509.26507v1.

[4] Engdahl et al. *BDH-CQ: In-Context Learning with Recurrent Latent Reasoning* (2026 report), section 3. arXiv:2608.09888v1.

[5] MemoryForge, implementation f3ec52e. Source, raw phase2_demo/phase3_evidence JSON, tests and technical walkthrough: github.com/deeps2710/MemoryForge.

[6] Behrouz et al. *Titans: Learning to Memorize at Test Time* (first submitted 2024-12-31), section 3. arXiv:2501.00663v1.

Codex assisted writing and verification. Automated checks do not establish human review. The team remains responsible for understanding and defending the code, claims and citations.